

CG作业2报告

李惟乐 PB23050929

内容

- 算法
 - IDW算法
 - RBF算法
 - 填补缝隙
- 结果

算法原理

Input: 给定 n 对控制点 $(\mathbf{p}_i, \mathbf{q}_i)$, 其中 $\mathbf{p}_i, \mathbf{q}_i \in \mathbb{R}^2$, $i = 1, 2, \dots, n$,

Output: 插值映射 $f: \mathbb{R}^2 \rightarrow \mathbb{R}^2$, 满足

$$f(\mathbf{p}_i) = \mathbf{q}_i, \quad \text{for } i = 1, 2, \dots, n.$$

IDW算法插值

数学原理

假设插值映射 f 具有如下加权平均的形式

$$f(\mathbf{p}) = \sum_{i=1}^n w_i(\mathbf{p}) f_i(\mathbf{p}),$$

其中 f_i 是在插值节点 $\mathbf{p}_i$ 处的局部近似 (local approximation), 满足在 $(\mathbf{p}_i, \mathbf{q}_i)$ 的插值性质 $f_i(\mathbf{p}_i) = \mathbf{q}_i$ 。权重函数 w_i 满足非负性和归一性

$$\sum_{i=1}^n w_i = 1, \text{ and } w_i \geq 0, \text{ for } i = 1, 2, \dots, n$$

且其在 $\mathbf{p}_i$ 处 $w_i(\mathbf{p}_i) = 1$.

对于任意的 i , 我们取如下形式的 w_i 和 f_i :

- 权重 $w_i: \mathbb{R}^2 \rightarrow \mathbb{R}$ 形如

$$w_i(\mathbf{p}) = \frac{\sigma_i(\mathbf{p})}{\sum_{j=1}^n \sigma_j(\mathbf{p})}$$

其中

$$\sigma_i(\mathbf{p}) = \frac{1}{\|\mathbf{p} - \mathbf{p}_i\|^\mu}$$

$\mu > 1$ (比如可以取 $\mu = 2$) , 在 $\mathbf{p} = \mathbf{p}_i$ 时 w_i 为1。

- 映射 $f_i: \mathbb{R}^2 \rightarrow \mathbb{R}^2$ 形如

$$f_i(\mathbf{p}) = \mathbf{q}_i + \mathbf{T}_i(\mathbf{p} - \mathbf{p}_i)$$

其中 $\mathbf{T}_i$ 是一个 2×2 矩阵, 通过最小化下面的能量决定 (最小二乘问题) :

$$E_i(\mathbf{T}_i) = \sum_{j=1, j \neq i}^n \sigma_i(\mathbf{p}_j) \|\mathbf{q}_i + \mathbf{T}_i(\mathbf{p}_j - \mathbf{p}_i) - \mathbf{q}_j\|^2.$$

求解这个最小二乘问题, 可以对矩阵 $\mathbf{T}_i$ 的各个分量求偏导数, 令偏导数等于 0, 得到一个关于 $\mathbf{T}_i$ 的方程组, 可以

写作:

$$\frac{\partial E_i}{\partial \mathbf{T}_i} = \sum_{j=1, j \neq i}^n 2 \cdot \sigma_i(\mathbf{p}_j) (\mathbf{q}_i + \mathbf{T}_i(\mathbf{p}_j - \mathbf{p}_i) - \mathbf{q}_j) \cdot (\mathbf{p}_j - \mathbf{p}_i)^\top = \mathbf{0}$$

即求解如下的线性方程组

$$\mathbf{T}_i \mathbf{A} = \mathbf{B}$$

其中

$$\mathbf{A} = \sum_{j=1, j \neq i}^n \sigma_i(\mathbf{p}_j) (\mathbf{p}_j - \mathbf{p}_i) \cdot (\mathbf{p}_j - \mathbf{p}_i)^\top,$$

$$\mathbf{B} = \sum_{j=1, j \neq i}^n \sigma_i(\mathbf{p}_j) (\mathbf{q}_j - \mathbf{q}_i) \cdot (\mathbf{p}_j - \mathbf{p}_i)^\top.$$

确定 f_i 和 w_i 之后, 就得到了插值映射 f .

编程

IDW类定义如下:

C IDWWarper	
● IDWWarper(const std::vector<ImVec2> & start_points, const std::vector<ImVec2> & end_points) : void ● ~IDWWarper() constexpr = default : void	
■ get_End_vector() : void ■ get_Matrix_A(int i) const : Matrix2d ■ get_Matrix_B(int i) const : Matrix2d ■ get_Matrix_T_list() : void ■ get_Start_vector() : void ■ get_f_i(int i, const Vector2d & p) const : Vector2d ■ get_final_func(const Vector2d & p) const : Vector2d ■ get_omega_i(int i, const Vector2d & p) const : double ■ get_sigma_i(int i, const Vector2d & p) const : double ● warp(int x, int y) const : std::pair<int,int>	
□ T_list : vector<Matrix2d> □ end_list : vector<Vector2d> □ start_list : vector<Vector2d>	

主体设计思路是：在传入start_points与end_points之后，优先将能确定的参数都先定出来，避免输入一个像素点就要重新生成一遍插值函数。因此额外在IDWWarper子类中设置了三个成员，分别存储矩阵 T_i ，起点和终点向量列表。

红色标注的private成员函数都是出于拆分任务，分散复杂度的考虑。在求解矩阵 T_i 时，涉及求矩阵 A 的逆矩阵，为提高运算稳定性，额外加入了一个正则参数 lambda

```
void IDWWarper::get_Matrix_T_list()
{
    T_list.resize(start_points_.size());

    Matrix2d A, B, T;
    A = B = T = Matrix2d::Zero();

    double lambda = 1e-5;    //regularization

    for (int i = 0; i < start_points_.size(); ++i)
    {
        A = get_Matrix_A(i) + lambda * Matrix2d::Identity();
        B = get_Matrix_B(i);

        T_list[i] = B * A.inverse();
    }
}
```

RBF算法插值

数学原理

假设所求的插值函数 f 是如下径向部分加上仿射部分的形式

$$f(\mathbf{p}) = A(\mathbf{p}) + R(\mathbf{p})$$

其中仿射变换部分满足

$$A(\mathbf{p}) = \mathbf{A}\mathbf{p} + \mathbf{b}$$

径向部分由若干个**径向**基函数组合而成

$$R(\mathbf{p}) = \sum_{i=1}^n \alpha_i g_i(\|\mathbf{p} - \mathbf{p}_i\|), \quad \alpha_i \in \mathbb{R}^2$$

这里的径向基函数也有较大的选取自由，可以取

$$g_i(d) = (d^2 + r_i^2)^{\mu/2}, \quad r_i = \min_{j \neq i} \|\mathbf{p}_i - \mathbf{p}_j\|.$$

上述映射 f 有 $2(n+3)$ 个自由度（矩阵 $\mathbf{A}$ ，向量 $\mathbf{b}$ ，以及 n 个二维向量 α_i ），插值条件

$$f(\mathbf{p}_j) = \sum_{i=1}^n \alpha_i g_i(\|\mathbf{p}_j - \mathbf{p}_i\|) + \mathbf{A}\mathbf{p}_j + \mathbf{b} = \mathbf{q}_j, \quad j = 1, \dots, n.$$

提供了 $2n$ 个约束。有以下的求解方法：

- 直接取 $\mathbf{A} = \mathbf{I}$, $\mathbf{b} = \mathbf{0}$ 是恒同映射，剩下的 $2n$ 个变量通过求解方程组确定；
- 如果提供了一个点，可以选取 $\mathbf{A}(\mathbf{p})$ 为平移变换， $\mathbf{A} = \mathbf{I}$, $\mathbf{b} = \mathbf{q}_1 - \mathbf{p}_1$ ；
- 如果提供了两个点，可以选取 $\mathbf{A}(\mathbf{p})$ 为平移+缩放；
- 一般地，如果提供了至少三个点，可以通过求解最小二乘问题确定仿射变换

$$\min \sum_{i=1}^n \|\mathbf{A}\mathbf{p}_i + \mathbf{b} - \mathbf{q}_i\|^2.$$

- 也可以补充约束求解所有 $2(n+3)$ 个变量

$$\begin{pmatrix} \mathbf{p}_1 & \cdots & \mathbf{p}_n \\ 1 & \cdots & 1 \end{pmatrix}_{3 \times n} \begin{pmatrix} \alpha_1^\top \\ \vdots \\ \alpha_n^\top \end{pmatrix}_{n \times 2} = \mathbf{0}_{3 \times 2}.$$

此处采用的是最后一种求解方法，通过令仿射变换部分与径向部分正交，得到另外的6个约束

经过变换后可以得到以下线性方程组

$$\mathbf{M}\mathbf{x} = \mathbf{v}$$

其中矩阵 $\mathbf{M}$ 有以下分块：

$$\mathbf{M} = \begin{bmatrix} \mathbf{M}_{affine} & \mathbf{M}_{radial} \\ \mathbf{0} & \mathbf{M}_{constr} \end{bmatrix}$$

即将矩阵 $\mathbf{M}$ 拆分为仿射矩阵块，径向矩阵块，额外约束矩阵块。其维数分别为 $2n \times 6$, $2n \times 2n$, $6 \times 2n$

待求解的向量 $\mathbf{x}$ ：

$$\mathbf{x} = [A_{11} \ A_{12} \ A_{21} \ A_{22} \ b_x \ b_y \ \alpha_{1x} \ \alpha_{1y} \ \dots \ \alpha_{nx} \ \alpha_{ny}]^T$$

右侧向量 $\mathbf{v}$ ：

$$\mathbf{v} = [q_{1x} \ q_{1y} \ \dots \ q_{nx} \ q_{ny} \ 0 \ 0 \ 0 \ 0 \ 0 \ 0]^T$$

求解该线性方程组之后，就可以从向量 x 中得到仿射所需的矩阵 A 和偏置向量 b 的值，也可以得知权重系数向量 α_i 的值。

编程

RBF类定义如下：

C RBFWarper

●

RBFWarper(const std::vector<ImVec2> & start_points, const std::vector<ImVec2> & end_points) : void

●

~RBFWarper() = default : void

■

Build_Matrix_M() : void

■

Solve_Equation() : void

■

get_End_vector() : void

■

get_Start_vector() : void

■

get_final_func(const Vector2d & p) const : Vector2d

■

get_g_d(const int & i, const double & d) const : double

■

get_r_list() : void

●

warp(int x, int y) const : std::pair<int,int>

□

Matrix_A : Matrix2d

□

Matrix_M : MatrixXd

□

alpha_list : vector<Vector2d>

□

bias : Vector2d

□

end_list : vector<Vector2d>

□

r_list : vector<double>

□

start_list : vector<Vector2d>

□

v : VectorXd

设计思路是类似的，也是参数一进来就把能初始化的全部初始化，避免重复运算。这里把 r_i 的值都算出来组织为向量，便于需要计算 $g_i(d)$ 时直接调用。

填补缝隙

缝隙的出现几乎是必然的，因为插值函数的映射以及计算过程中对小数的阶段，都可能会导致不同位置的点映射到了同一像素，甚至超出边界。因此需要填补缝隙。

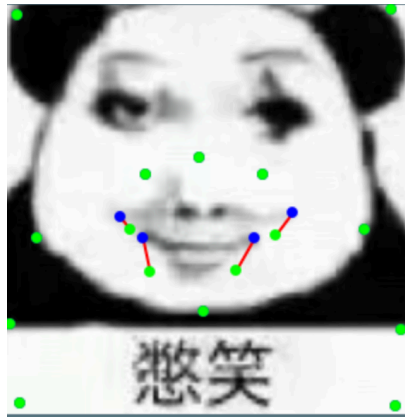
这里采用的填补缝隙方法是利用ANN库搜索黑洞点附近的有效点，将其值直接复制过来。程序实现上大致分为三步：

- 记录黑洞点和有效点，以及有效点的像素数据，组织为三个向量
- 利用ANN库遍历黑洞点，为其找到附近的有效点
- 如果其距离有效点距离小于某个设定的值，就将有效点的值直接复制过来

增加距离判断是为了避免图像变形后正常出现的大片黑色区域也被填充。

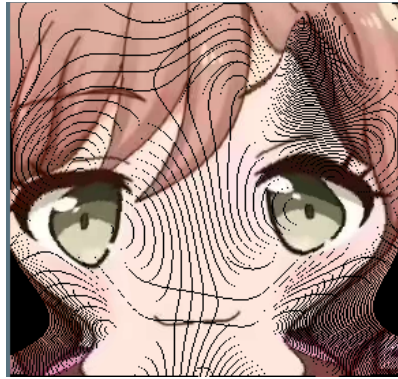
结果

IDW展示（控制点展示、无补缝展示、有补缝展示）

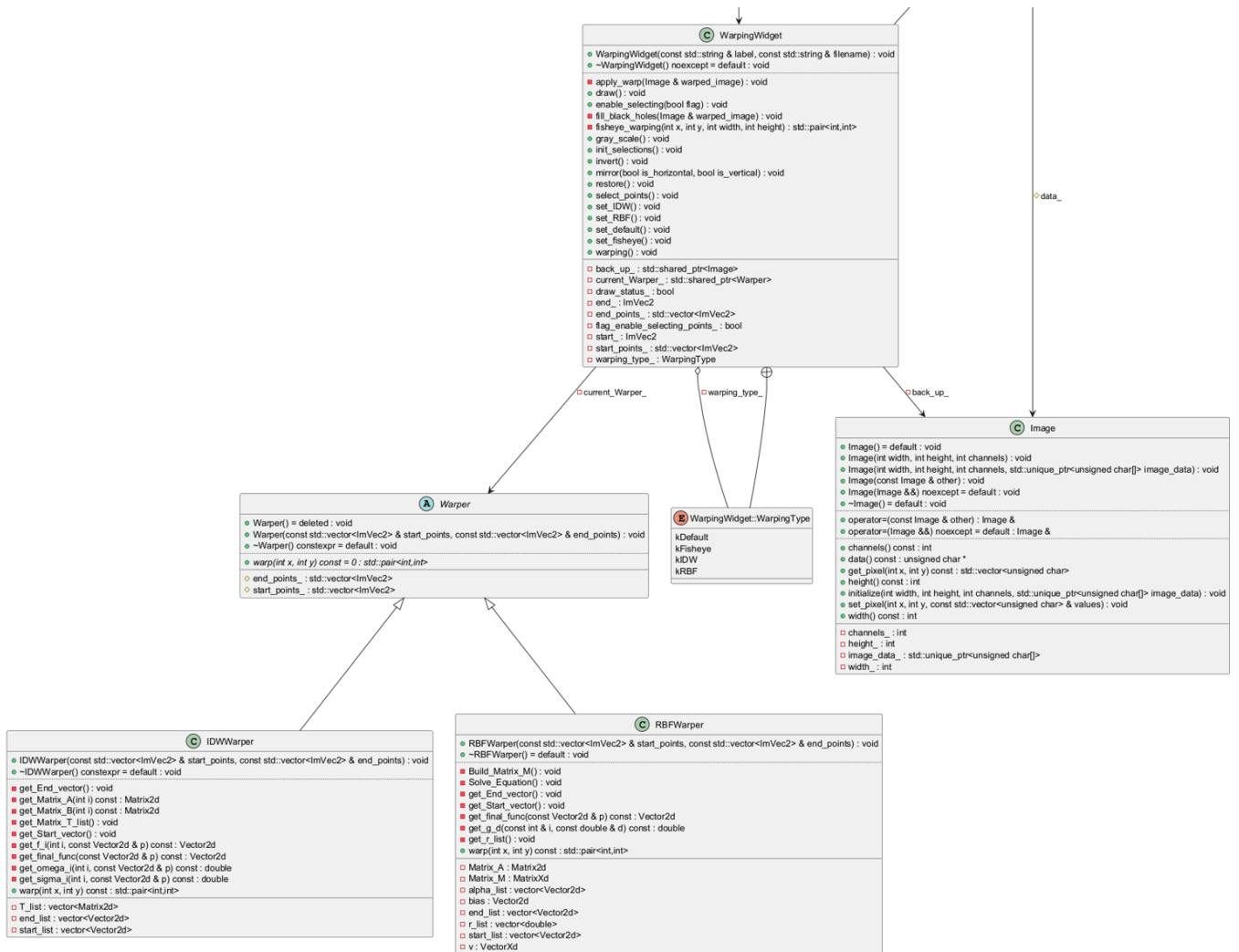


RBF展示 (控制点展示、无补缝展示、有补缝展示)





总体类图展示:



debug过程与反思

在编写程序过程中，出现过以下问题：

- 函数中未采用引用参数，导致对拷贝进行了操作，而非对目标本身
- 在编写RBF算法时，最后的返回值仅有仿射变换部分，径向部分遗漏了，导致整体效果类似线性变换

比较严重的问题应该是对程序的理解。开始时对程序要达成的效果和交互方式没有明确的认知，导致在程序已经可以正常运行的情况下，仍然以为存在bug。在选中多个固定点之后才发现程序已经达到目的，白白浪费了大量时间。之后编写程序时应该先对算法和程序所能达到的效果有心理预期，否则容易白费工夫。

另外目前该程序还可以进行如下改进：

- 运行效率较低，特别是在引入补缝算法后，运行速度大大降低。应当进一步提升效率；
- 鱼眼未封装成Warper下的一个类，可以尝试将修改为Warper下的一个子类；
- 神经网络方式尚未实现，之后若需实现，可将其也作为Warper的一个子类，接口对应即可。